BANGLA COMPLETE GUIDE

Laravel Eloquent ORM

সম্পূর্ণ বাংলা গাইড

Story-based Learning | Real Project | 200+ Examples

16	200+	Real
Modules	Code Examples	Project

এই বইয়ে যা শিখবে:

- One-to-One, One-to-Many, Many-to-Many, Polymorphic Relationship
- N+1 Problem সমাধান ও Eager Loading Optimization
- Scope, Accessor, Mutator, Soft Delete, Observer
- Real E-commerce Project সম্পূর্ণ Codebase
- Story দিয়ে বোঝানো — যেন মনে থাকে সারাজীবন

Beginner থেকে Advanced

PHP ও Laravel basic জানলেই শুরু করা যাবে

Story-based Learning

গল্পের মাধ্যমে concept বোঝানো — সহজে মনে থাকে

সূচিপত্র (Table of Contents)

Module 0 ভূমিকা ও শুরুর কথা

– বইটি কীভাবে পড়বে
– Laravel Version
– Environment Setup

Module 1 ORM ও Eloquent Basics

– ORM কী
– Model তৈরি
– Fillable vs Guarded
– সম্পূর্ণ CRUD
– Convention

Module 2 One-to-One Relationship

– hasOne / belongsTo
– User → Profile story
– Custom Foreign Key
– Eager Loading

Module 3 One-to-Many Relationship

– hasMany / belongsTo
– User → Posts story
– Query Builder vs Property
– Real Examples

Module 4 Many-to-Many Relationship

– belongsToMany
– Post ↔ Tags story
– attach/detach/sync
– Pivot Extra Data

Module 5 Has One & Many Through

– hasManyThrough story
– Country → User → Post
– Custom Keys

Module 6 Polymorphic Relationship

– morphMany story
– One-to-One Poly
– One-to-Many Poly
– Many-to-Many Poly

Module 7 Eager Loading ও N+1 Problem

– N+1 গল্প দিয়ে বোঝানো
– with() nested
– withCount/Sum/Avg

– Performance Tips

Module 8 Querying Relationships

– whereHas / has
– doesntHave
– withSum/withAvg
– Advanced Filters

Module 9 Accessors, Mutators ও Casts

– Accessor story
– Mutator story
– $casts
– Custom Cast

Module 10 Scopes

– Local Scope
– Global Scope
– Named Scope chaining

Module 11 Soft Deletes

– Soft Delete story
– Restore
– Force Delete
– withTrashed

Module 12 Model Events ও Observer

– Lifecycle Events
– Observer class
– Real use cases

Module 13 Factory ও Seeder

– Factory তৈরি
– Relationship সহ Seeder
– State
– Faker tips

Module 14 API Resource

– JsonResource
– conditional loading
– Collection
– whenLoaded

Module 15 Real Project: E-Commerce

– Full DB Design
– All Models
– Controllers
– Order System
– API

Module 16 Best Practices ও Cheat Sheet

- Naming Convention
- Performance Table
- Common Mistakes
- Quick Reference

ভূমিকা — বইটি পড়ার আগে

📚 একটি ছোট গল্প — কেন এই বই লেখা হলো

রাকিব একজন PHP developer। সে Laravel শিখছে। একদিন তার বস বললেন, "রাকিব,
sale report এ প্রতিটি sale এর সাথে product এর brand name দেখাও।"
রাকিব ঘণ্টার পর ঘণ্টা চেষ্টা করলো। Page load হতে ৮ সেকেন্ড লাগছে।
কারণ — সে প্রতিটি sale এর জন্য আলাদা আলাদা query চালাচ্ছিল।
যদি সে Eloquent Relationship এবং Eager Loading জানতো,
তাহলে এই কাজটি হতো ২ লাইনে, ০.১ সেকেন্ডে।

এই বইটি রাকিবের মতো সব developers এর জন্য। Story দিয়ে শেখা,
Code দিয়ে practice করা — এটাই আমাদের পথ।

বইটি কীভাবে পড়বে

প্রতিটি Module এর শুরুতে একটি গল্প আছে — এটা concept কে real-life এর সাথে connect করে।
এরপর theory, তারপর code example। প্রতিটি Module শেষে Practice section আছে।

Icon	মানে	কখন দেখবে
📚 গল্প	Story-based explanation	Concept বোঝার আগে
💡 টিপস	Helpful tips	Important shortcut
⚠ সাবধান	Warning	ভুল যেন না হয়
📌 নোট	Extra info	জেনে রাখা ভালো
🎯 Practice	Exercise	পড়া শেষে করো

📌 **নোট:** Laravel 10+ ব্যবহার করা হয়েছে। PHP 8.1+ এবং Composer install থাকতে হবে।

Module
01

ORM ও Eloquent Basics

শূন্য থেকে শুরু

Model | CRUD | Convention | Fillable | Timestamps

📚 গল্প — Database ছাড়া কি কাজ চলে?

রফিক একটি school এর software বানাচ্ছে। তাকে students এর information save করতে হবে।

সে প্রথমে raw SQL লিখল: →INSERT INTO students...", →SELECT * FROM students WHERE..."

Code বড় হওয়ার সাথে সাথে SQL গুলো everywhere ছড়িয়ে গেল। Bug ধরা কঠিন হয়ে গেল।

তার senior বললেন: →ORM ব্যবহার করো — তাহলে PHP object দিয়েই database কাজ হবে।→

ORM শেখার পর রফিক বুঝলো — এটাই সঠিক পথ। Code clean, readable এবং secure।

১.১ ORM কী? (Object Relational Mapping)

ORM = Object Relational Mapping। এটি একটি technique যা database table কে PHP Object হিসেবে দেখে।

ছাড়া ORM (Raw SQL)	ORM দিয়ে (Eloquent)
SELECT * FROM users	User::all()
SELECT * FROM users WHERE id=1	User::find(1)
INSERT INTO users (name) VALUES ('Karim')	User::create(['name'=>'Karim'])
UPDATE users SET name='Rahim' WHERE id=1	User::where('id',1)->update(['name'=>'Rahim'])
DELETE FROM users WHERE id=1	User::find(1)->delete()

💡 **সুবিধা** ORM ব্যবহার করলে SQL injection কমে, code readable হয়, এবং database change করলেও code বেশি change লাগে না।

১.২ Model তৈরি করা

```
# শুধু Model
php artisan make:model Student

# Model + Migration
php artisan make:model Student -m

# Model + Migration + Controller + Factory + Seeder (সব একসাথে!)
```

```
php artisan make:model Student -a

# তৈরি হওয়া file দেখো: app/Models/Student.php
```

১.৩ Table Naming Convention

Laravel automatically নিচের নিয়ম মেনে table খোঁজে:

Model Name	Table Name (Auto)	Custom করতে
User	users	protected $table = 'users';
Post	posts	protected $table = 'blog_posts';
ProductCategory	product_categories	protected $table = 'prod_cats';
OrderDetail	order_details	যেকোনো নাম দেওয়া যায়

১.৪ Fillable vs Guarded

Mass assignment protection — একসাথে অনেক field fill করার নিরাপত্তা:

```
class Student extends Model
{
    // শুধু এই fields fill করা যাবে
    protected $fillable = ['name', 'email', 'class', 'roll'];

    // অথবা, এই fields ছাড়া সব fill করা যাবে
    // protected $guarded = ['id'];

    // Timestamps auto manage (created_at, updated_at)
    // Disable করতে:
    // public $timestamps = false;

    // Custom timestamp column name:
    // const CREATED_AT = 'create_time';
    // const UPDATED_AT = 'update_time';
}
```

⚠ **সাবধান:** কখনো $guarded = [] করো না Production এ এটা সব fields কে fillable করে দেয়, যা security risk।

১.৫ সম্পূর্ণ CRUD — Real Example

Create

```
// ১. create() — সবচেয়ে সহজ
$student = Student::create([
    'name'  => 'Karim Hossain',
    'email' => 'karim@school.com',
    'class' => 10,
    'roll'  => 1,
]);

// ২. new + save()
$student = new Student();
$student->name  = 'Rahim';
$student->email = 'rahim@school.com';
$student->save();

// ৩. firstOrCreate — না থাকলে create, থাকলে শুধু return
$student = Student::firstOrCreate(
    ['email' => 'karim@school.com'],        // where condition
    ['name' => 'Karim', 'class' => 10]       // extra fields
);

// ৪. updateOrCreate — থাকলে update, না থাকলে create
$student = Student::updateOrCreate(
    ['roll' => 1, 'class' => 10],
    ['name' => 'Updated Karim']
);
```

Read

```
// সব students
$all = Student::all();

// নির্দিষ্ট id
$s = Student::find(1);
$s = Student::findOrFail(1);  // না পেলে 404

// Where condition
$s = Student::where('class', 10)->get();
$s = Student::where('class', 10)->where('roll', 1)->first();

// Multiple where
$s = Student::where('class', '>=', 9)
             ->where('status', 'active')
             ->orderBy('name', 'asc')
```

```
            ->limit(20)
            ->get();

// নির্দিষ্ট columns
$s = Student::select('id', 'name', 'class')->get();

// Paginate
$s = Student::paginate(15);

// whereIn
$s = Student::whereIn('class', [9, 10, 11])->get();

// whereBetween
$s = Student::whereBetween('roll', [1, 50])->get();

// Search (LIKE)
$s = Student::where('name', 'LIKE', '%karim%')->get();
```

Update

```
// ১. find + modify + save
$student = Student::find(1);
$student->name = 'New Name';
$student->save();

// ২. update() — bulk update
Student::where('class', 10)->update(['status' => 'graduated']);

// ৩. increment/decrement
Student::where('id', 1)->increment('score', 5);   // score += 5
Student::where('id', 1)->decrement('score', 2);   // score -= 2

// ৪. fill + save
$student = Student::find(1);
$student->fill(['name' => 'Karim', 'class' => 11]);
$student->save();
```

Delete

```
// ১. find + delete
Student::find(1)->delete();

// ২. where + delete
```

```
Student::where('class', 12)->where('status', 'graduated')->delete();


// ৩. destroy — id দিয়ে সরাসরি
Student::destroy(1);
Student::destroy([1, 2, 3]);
```

🎯 Practice করো

1. একটি Product model তৈরি করো (name, price, stock, category)
2. 5টি Product create করো, তারপর সব read করো
3. price > 100 এমন products এর stock বাড়াও 10 করে
4. id = 3 এর Product delete করো

Module
02

One-to-One Relationship

একটির সাথে ঠিক একটি

hasOne | belongsTo | Inverse | Custom Keys

📚 গল্প — জাতীয় পরিচয়পত্র

বাংলাদেশে প্রতিটি নাগরিকের শুধুমাত্র একটি NID (জাতীয় পরিচয়পত্র) থাকতে পারে।

একটি NID শুধু একজন নাগরিকের।

এটাই One-to-One Relationship←

Software এ: একজন User থাকে, তার একটি Profile থাকে।

Profile জানে সে কোন User এর — user_id দিয়ে।

User জানে তার Profile আছে — hasOne() দিয়ে।

Profile জানে সে কার — belongsTo() দিয়ে।

২.১ Database Design

```
// users table (আগে থেকেই আছে)
// profiles table migration
Schema::create('profiles', function (Blueprint $table) {
    $table->id();
    $table->unsignedBigInteger('user_id')->unique(); // unique = one-to-one
enforce
    $table->string('phone')->nullable();
    $table->string('avatar')->nullable();
    $table->text('bio')->nullable();
    $table->date('dob')->nullable();
    $table->string('address')->nullable();
    $table->string('city')->nullable();
    $table->timestamps();

    $table->foreign('user_id')
          ->references('id')->on('users')
          ->onDelete('cascade'); // User delete হলে Profile ও delete
});
```

২.২ Model এ Relationship

```
// app/Models/User.php
class User extends Model
{
    protected $fillable = ['name', 'email', 'password'];

    /**
     * User এর একটি Profile আছে
     * hasOne(Related, foreignKey, localKey)
     */
    public function profile()
    {
        return $this->hasOne(Profile::class);
        // Laravel automatically খুঁজবে: profiles.user_id
    }

    // Custom key দিতে চাইলে:
    // return $this->hasOne(Profile::class, 'custom_user_id', 'id');
}

// app/Models/Profile.php
class Profile extends Model
{
    protected $fillable =
['user_id','phone','avatar','bio','dob','address','city'];

    /**
     * Profile টি একজন User এর অন্তর্গত
     */
    public function user()
    {
        return $this->belongsTo(User::class);
        // Laravel automatically খুঁজবে: profiles.user_id
    }
}
```

২.৩ Data Create করা

```
// User তৈরি করো
$user = User::create([
    'name'     => 'Karim Hossain',
    'email'    => 'karim@mail.com',
    'password' => bcrypt('password123'),
]);
```

```
// তার Profile তৈরি করো (user_id auto set হবে)
$user->profile()->create([
    'phone'   => '01711223344',
    'bio'     => 'PHP Developer from Dhaka',
    'address' => 'Mirpur, Dhaka',
    'city'    => 'Dhaka',
]);

// অথবা associate() ব্যবহার করে
$profile = new Profile(['phone' => '01711223344']);
$profile->user()->associate($user);
$profile->save();
```

২.৪ Data Access করা — সব পদ্ধতি

```
// ১. User থেকে Profile
$user    = User::find(1);
$profile = $user->profile;   // Profile object (অথবা null)

echo $user->name;
echo $user->profile->phone;
echo $user->profile->city;

// ২. Profile থেকে User
$profile = Profile::find(1);
$user    = $profile->user;   // User object

echo $profile->phone;
echo $profile->user->name;
echo $profile->user->email;

// ৩. Profile আছে কিনা check
if ($user->profile) {
    echo 'Profile আছে: ' . $user->profile->phone;
} else {
    echo 'Profile এখনো তৈরি হয়নি';
}

// ৪. Eager Loading সহ সব Users
$users = User::with('profile')->get();
foreach ($users as $user) {
    $phone = $user->profile?->phone ?? 'N/A';
    echo $user->name . ' — ' . $phone;
```

```
}
```

২.৫ Update ও Delete

```
// Profile Update
$user = User::find(1);
$user->profile->update(['phone' => '01722334455', 'city' => 'Chittagong']);

// অথবা updateOrCreate (profile না থাকলে create, থাকলে update)
$user->profile()->updateOrCreate(
    ['user_id' => $user->id],
    ['phone' => '01733445566', 'bio' => 'Updated bio']
);

// Profile Delete
$user->profile->delete();
// users table এ cascade থাকলে user delete হলে profile ও যাবে
```

💡 **Null Safety** $user->profile?->phone এভাবে লিখলে profile null হলেও error আসবে না। PHP 8+ এ এটা অনেক কাজে লাগে।

🎯 **Practice করো**

5. Company ও CompanyDetails Model তৈরি করো (hasOne/belongsTo)
6. 5টি Company তৈরি করো, প্রতিটির Details তৈরি করো
7. Eager Loading দিয়ে সব Company তাদের Details সহ দেখাও
8. id=2 Company এর details update করো

Module
03

One-to-Many Relationship

একটির সাথে অনেক

hasMany | belongsTo | Query Builder | Sorting | Count

📚 গল্প — একজন শিক্ষক ও তার ছাত্রছাত্রী

একজন Teacher অনেকজন Student পড়াতে পারেন।

কিন্তু একজন Student এর primary Teacher একজনই।

Teacher Model বলে: 'আমার অনেক Students আছে' — hasMany()

Student Model বলে: 'আমি একজন Teacher এর অন্তর্গত' — belongsTo()

Database এ students table এ teacher_id column থাকে।

এই teacher_id দিয়েই relationship বোঝা যায়।

আরো উদাহরণ:

একটি Blog Category → অনেক Blog Posts

একটি Brand → অনেক Products

একটি Order → অনেক Order Items

৩.১ Database Design

```
// teachers table
Schema::create('teachers', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->string('subject');
    $table->string('email')->unique();
    $table->timestamps();
});

// students table
Schema::create('students', function (Blueprint $table) {
    $table->id();
    $table->unsignedBigInteger('teacher_id');  // FK
    $table->string('name');
    $table->string('email')->unique();
    $table->integer('class');
```

```
    $table->integer('roll');
    $table->decimal('gpa', 3, 2)->default(0);
    $table->timestamps();

    $table->foreign('teacher_id')
          ->references('id')->on('teachers')
          ->onDelete('restrict'); // Teacher delete হলে error আসবে
});
```

৩.২ Models

```
// Teacher Model
class Teacher extends Model
{
    protected $fillable = ['name', 'subject', 'email'];

    // একজন Teacher এর অনেক Students আছে
    public function students()
    {
        return $this->hasMany(Student::class);
    }

    // শুধু ভালো GPA Students (Conditional relationship)
    public function topStudents()
    {
        return $this->hasMany(Student::class)
                    ->where('gpa', '>=', 4.0)
                    ->orderBy('gpa', 'desc');
    }

    // সর্বশেষ student
    public function latestStudent()
    {
        return $this->hasOne(Student::class)->latestOfMany();
    }
}

// Student Model
class Student extends Model
{
    protected $fillable = ['teacher_id','name','email','class','roll','gpa'];

    // একজন Student একজন Teacher এর অন্তর্গত
    public function teacher()
```

```
    {
        return $this->belongsTo(Teacher::class);
    }
}
```

৩.৩ Data Create

```
// Teacher তৈরি করো
$teacher = Teacher::create([
    'name'    => 'Mr. Alam',
    'subject' => 'Mathematics',
    'email'   => 'alam@school.com',
]);

// Teacher এর জন্য Students তৈরি
$teacher->students()->create([
    'name'  => 'Karim',
    'email' => 'karim@student.com',
    'class' => 10,
    'roll'  => 1,
    'gpa'   => 4.5,
]);

// একসাথে অনেক students (createMany)
$teacher->students()->createMany([
    ['name'=>'Rahim', 'email'=>'rahim@s.com', 'class'=>10, 'roll'=>2,
'gpa'=>4.2],
    ['name'=>'Jamal', 'email'=>'jamal@s.com', 'class'=>10, 'roll'=>3,
'gpa'=>3.8],
    ['name'=>'Sadia', 'email'=>'sadia@s.com', 'class'=>10, 'roll'=>4,
'gpa'=>4.9],
]);
```

৩.৪ Data Access — বিভিন্ন পদ্ধতি

```
// ১. সব Students (Property — lazy load)
$teacher  = Teacher::find(1);
$students = $teacher->students; // Collection

// ২. Query Builder দিয়ে filter
$topStudents = $teacher->students()
                        ->where('gpa', '>=', 4.0)
                        ->orderBy('gpa', 'desc')
                        ->get();
```

```
// ৩. Count
$count = $teacher->students()->count();

// ৪. exists()
if ($teacher->students()->exists()) {
    echo 'Students আছে';
}

// ৫. Student থেকে Teacher
$student = Student::find(1);
echo $student->teacher->name;
echo $student->teacher->subject;

// ৬. Eager Loading — সব Teacher তাদের Students সহ
$teachers = Teacher::with('students')->get();
foreach ($teachers as $teacher) {
    echo $teacher->name . ' — ' . $teacher->students->count() . ' students';
    foreach ($teacher->students as $student) {
        echo '  ' . $student->name . ' (GPA: ' . $student->gpa . ')';
    }
}

// ৭. withCount — Student count সহ Teacher
$teachers = Teacher::withCount('students')->get();
foreach ($teachers as $t) {
    echo $t->name . ' এর ' . $t->students_count . ' জন ছাত্র আছে';
}
```

৩.৫ Advanced Query

```
// Conditional Eager Loading
$teachers = Teacher::with(['students' => function($query) {
    $query->where('gpa', '>=', 3.5)
          ->orderBy('gpa', 'desc');
}])->get();

// Nested Data Access
// সব students তাদের teacher এর subject সহ
$students = Student::with('teacher:id,name,subject')
                   ->where('class', 10)
                   ->orderBy('roll')
                   ->get();
```

```
foreach ($students as $s) {
    echo $s->name . ' — ' . $s->teacher->subject;
}
```

৩.৬ property vs method — গুরুত্বপূর্ণ পার্থক্য!

পদ্ধতি	কী হয়	কখন ব্যবহার করবে
$teacher->students	সব data একসাথে load, cache হয়	Loop এ access করলে
$teacher->students()	Query Builder return করে	Filter/Sort/Count এর আগে
$teacher->students()->count()	Database এ count query	কেবল count দরকার হলে
$teacher->students->count()	PHP Collection এ count	Data already loaded হলে

⚠ **সাবধান:** Loop এর ভেতরে $teacher->students() (method) call করবে না। এটা প্রতিবার নতুন query করে। পরিবর্তে $teacher->students (property) ব্যবহার করো অথবা Eager Loading।

🎯 Practice করো

9. Category ও Post model তৈরি করো (hasMany/belongsTo)
10. 3টি Category তৈরি করো, প্রতিটিতে 5টি Post
11. withCount দিয়ে প্রতিটি Category এর Post সংখ্যা দেখাও
12. শুধু published posts আছে এমন Categories বের করো

Module
04

Many-to-Many Relationship

অনেক ↔ অনেক

belongsToMany | Pivot Table | attach | detach | sync

📚 গল্প — Social Media Friends

Facebook এ Karim এর অনেক Friends আছে।

Rahim ও অনেকজনের Friend।

Karim, Rahim এর Friend — এবং Rahim, Karim এর Friend।

এটাই Many-to-Many← একজনের অনেক Friend থাকতে পারে।

একজন অনেকজনের Friend হতে পারে।

Database এ আলাদা একটি 'friendships' table লাগে — Pivot Table।

user_id এবং friend_id — এই দুটো column দিয়ে relationship track হয়।

আরো উদাহরণ:

Student ↔ Course (একজন student অনেক course নিতে পারে)

Post ↔ Tag (একটি post এ অনেক tags, একটি tag অনেক posts এ)

User ↔ Role (একজন user এর অনেক role, একটি role এ অনেক user)

৪.১ Pivot Table বোঝা

Pivot Table হলো দুটো Model এর মাঝখানে থাকা table।এর নাম সাধারণত দুটো Model এর নাম alphabetical order এ।

```
// courses table
Schema::create('courses', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->string('code')->unique();
    $table->integer('credit_hours');
    $table->timestamps();
});

// Pivot table: course_student (alphabetical: c আগে, s পরে)
Schema::create('course_student', function (Blueprint $table) {
```

```
    $table->id();
    $table->unsignedBigInteger('course_id');
    $table->unsignedBigInteger('student_id');
    $table->decimal('marks', 5, 2)->nullable();   // extra pivot column
    $table->string('grade')->nullable();           // extra pivot column
    $table->enum('status', ['enrolled','passed','failed'])->default('enrolled');
    $table->timestamps();



$table->foreign('co
urse_id')->references('id')->on('courses')-
>onDelete('cascade');

$table->foreign('
student_id')->references('id')->on('s
tudents')->onDelete('cascade');
    $table->unique(['course_id', 'student_id']); // duplicate prevent
});
```

৪.২ Models

```
// Student Model
class Student extends Model
{
    public function courses()
    {
        return $this->belongsToMany(Course::class)
                    ->withPivot('marks', 'grade', 'status')  // extra pivot
columns
                    ->withTimestamps();                      // pivot এ
timestamps
    }


    // শুধু passed courses
    public function passedCourses()
    {
        return $this->belongsToMany(Course::class)
                    ->wherePivot('status', 'passed')
                    ->withPivot('marks', 'grade');
    }
}


// Course Model
class Course extends Model
{
    public function students()
    {
```

```
        return $this->belongsToMany(Student::class)
                    ->withPivot('marks', 'grade', 'status')
                    ->withTimestamps();
    }
}
```

৪.৩ attach — সম্পর্ক যোগ করা

```
$student = Student::find(1);

// একটি Course enroll করা
$student->courses()->attach(1);

// Pivot data সহ enroll
$student->courses()->attach(1, [
    'status' => 'enrolled',
]);

// একসাথে অনেক courses enroll
$student->courses()->attach([1, 2, 3]);

// প্রতিটির জন্য আলাদা pivot data
$student->courses()->attach([
    1 => ['status' => 'enrolled'],
    2 => ['status' => 'enrolled'],
    3 => ['status' => 'enrolled'],
]);
```

৪.৪ detach — সম্পর্ক সরানো

```
$student = Student::find(1);

// একটি Course remove
$student->courses()->detach(1);

// একাধিক
$student->courses()->detach([1, 2]);

// সব courses remove
$student->courses()->detach();
```

৪.৫ sync — সবচেয়ে কাজের method!

```
// sync — নির্দিষ্ট ids রাখো, বাকি সব সরাও
// Form submit এর পর use করার জন্য BEST
$student->courses()->sync([1, 3, 5]);
// আগে 1,2,3 ছিল — এখন 2 চলে যাবে, 5 নতুন যোগ হবে

// Pivot data সহ sync
$student->courses()->sync([
    1 => ['status' => 'passed', 'marks' => 85.5, 'grade' => 'A+'],
    3 => ['status' => 'enrolled'],
    5 => ['status' => 'enrolled'],
]);

// syncWithoutDetaching — আগেরটা রেখে নতুন যোগ করো
$student->courses()->syncWithoutDetaching([4, 5]);

// toggle — থাকলে সরাও, না থাকলে যোগ করো
$student->courses()->toggle([1, 2]);
```

৪.৬ Pivot Data Access

```
// Student এর সব Courses + marks
$student = Student::with('courses')->find(1);

foreach ($student->courses as $course) {
    echo $course->name;
    echo $course->pivot->marks;    // pivot data
    echo $course->pivot->grade;
    echo $course->pivot->status;
}

// Pivot data update করা
$student->courses()->updateExistingPivot(1, [
    'marks' => 92.0,
    'grade' => 'A+',
    'status' => 'passed'
]);
```

৪.৭ Query with Many-to-Many

```
// কোনো course এ enrolled আছে এমন Students
```

```
$students = Student::has('courses')->get();

// নির্দিষ্ট course এ enrolled Students
$students = Student::whereHas('courses', function($q) {
    $q->where('code', 'CSE101');
})->get();

// A+ grade পেয়েছে এমন Students
$students = Student::whereHas('courses', function($q) {
    $q->wherePivot('grade', 'A+');
})->with(['courses' => function($q) {
    $q->wherePivot('grade', 'A+')->withPivot('marks','grade');
}])->get();
```

🎯 Practice করো

13. User ও Role Model তৈরি করো (belongsToMany)
14. roles: admin, editor, viewer — তৈরি করো
15. 3 জন User তৈরি করো, attach দিয়ে roles assign করো
16. sync দিয়ে একজন User এর roles update করো
17. admin role আছে এমন সব Users বের করো

Module
05

Has One / Many Through

মধ্যবর্তী Model এর ভেতর দিয়ে

hasManyThrough | hasOneThrough | Bridge Model

📚 গল্প — দেশ, শহর ও মানুষ

Bangladesh এর অনেক City আছে।

প্রতিটি City তে অনেক User আছে।

এখন কেউ যদি জিজ্ঞেস করে:

'Bangladesh এর সব Users কারা?' — সরাসরি Bangladesh → User কোনো relation নেই।

কিন্তু Bangladesh → City → User — এই পথে যাওয়া যায়।

এটাই hasManyThrough←

তুমি City model ছাড়াই সরাসরি Country থেকে User পাবে।

৫.১ hasManyThrough

```
// countries table: id, name, code
// cities table:    id, country_id, name
// users table:     id, city_id, name, email

// Country Model
class Country extends Model
{
    // Country → City → User
    public function users()
    {
        return $this->hasManyThrough(
            User::class,     // চাই এটা
            City::class,     // এর ভেতর দিয়ে
            'country_id',    // City table এ Country এর FK
            'city_id',       // User table এ City এর FK
            'id',            // Country এর local key
            'id'             // City এর local key
        );
    }
}
```

```
// ব্যবহার
$bd      = Country::where('code','BD')->first();
$users   = $bd->users;                      // সব Bangladeshi users
$count   = $bd->users()->count();            // count
$dhaka   = $bd->users()->where('city_id', 5)->get();
```

৫.২ hasOneThrough

```
// Car → Engine → Mechanic
// (একটি Car এর একটি Engine, সেই Engine এর একজন Mechanic)

class Car extends Model
{
    public function mechanic()
    {
        return $this->hasOneThrough(
            Mechanic::class,
            Engine::class,
            'car_id',      // Engine table এ Car এর FK
            'engine_id',   // Mechanic table এ Engine এর FK
        );
    }
}
```

> 💡 **কখন ব্যবহার করবে** যখন A → B → C এই chain এ directly A থেকে C দরকার কিন্তু B model টা ব্যবহার করতে চাও না। hasManyThrough B কে skip করে A থেকে সরাসরি C দেয়।

Module
06

Polymorphic Relationship

একটি Model — অনেক ভিন্ন Model

morphTo | morphMany | morphOne | morphToMany

📚 গল্প — সবার জন্য একটি Comment system

Rahim একটি news portal বানাচ্ছে।

News Article এও comment করা যাবে।

Video তেও comment করা যাবে।

Photo তেও comment করা যাবে।

সে ভাবলো: 3টি আলাদা table বানাবো?

article_comments, video_comments, photo_comments?

তার senior বললেন: না← একটি comments table করো।

সেখানে রাখো: commentable_id (কোন record) এবং commentable_type (কোন Model)।

এটাই Polymorphic Relationship — অনেক flexible←

৬.১ One-to-Many Polymorphic

```
// comments table
Schema::create('comments', function (Blueprint $table) {
    $table->id();
    $table->unsignedBigInteger('user_id');
    $table->text('body');
    $table->unsignedBigInteger('commentable_id');   // কোন record এর id
    $table->string('commentable_type');             // কোন Model class
    $table->timestamps();

    // Example data:
    // commentable_id = 5, commentable_type = 'App\Models\Article'
    // commentable_id = 3, commentable_type = 'App\Models\Video'
});
```

```
// Comment Model
class Comment extends Model
{
    protected $fillable = ['user_id', 'body', 'commentable_id',
```

```
'commentable_type'];

    // কোন model এর comment সেটা বুঝবে (morphTo)
    public function commentable()
    {
        return $this->morphTo();
    }

    // Comment করেছে কে
    public function user()
    {
        return $this->belongsTo(User::class);
    }
}

// Article Model
class Article extends Model
{
    public function comments()
    {
        return $this->morphMany(Comment::class, 'commentable');
    }
}

// Video Model
class Video extends Model
{
    public function comments()
    {
        return $this->morphMany(Comment::class, 'commentable');
    }
}

// Photo Model
class Photo extends Model
{
    public function comments()
    {
        return $this->morphMany(Comment::class, 'commentable');
    }
}
```

৬.২ Polymorphic Data Create ও Access

```
// Article এ comment
$article = Article::find(1);
$article->comments()->create([
    'user_id' => auth()->id(),
    'body'    => 'চমৎকার আর্টিকেল!',
]);

// Video তে comment
$video = Video::find(1);
$video->comments()->create([
    'user_id' => auth()->id(),
    'body'    => 'অসাধারণ ভিডিও!',
]);

// Article এর সব comments
$article = Article::with('comments.user')->find(1);
foreach ($article->comments as $comment) {
    echo $comment->user->name . ': ' . $comment->body;
}

// Comment থেকে parent বের করা
$comment = Comment::find(1);
$parent  = $comment->commentable; // Article অথবা Video object return করবে
echo get_class($parent);           // 'App\Models\Article'
```

৬.৩ One-to-One Polymorphic (morphOne)

```
// images table
Schema::create('images', function (Blueprint $table) {
    $table->id();
    $table->string('url');
    $table->string('alt')->nullable();
    $table->unsignedBigInteger('imageable_id');
    $table->string('imageable_type');
    $table->timestamps();
});

// Image Model
class Image extends Model
{
    public function imageable() { return $this->morphTo(); }
}
```

```
// User Model
class User extends Model
{
    // User এর একটি Avatar Image
    public function avatar() { return $this->morphOne(Image::class,
'imageable'); }
}

// Article Model
class Article extends Model
{
    // Article এর একটি Featured Image
    public function featuredImage() { return $this->morphOne(Image::class,
'imageable'); }
}

// ব্যবহার
$user = User::find(1);
$user->avatar()->create(['url' => '/uploads/avatar.jpg']);

$article = Article::find(1);
$article->featuredImage()->create(['url' => '/uploads/featured.jpg', 'alt' =>
'Article image']);
```

৬.৪ Many-to-Many Polymorphic (morphToMany)

```
// tags table + taggables pivot table
Schema::create('taggables', function (Blueprint $table) {
    $table->id();
    $table->unsignedBigInteger('tag_id');
    $table->unsignedBigInteger('taggable_id');
    $table->string('taggable_type');
    $table->timestamps();
});

// Tag Model
class Tag extends Model
{
    public function articles() { return $this->morphedByMany(Article::class,
'taggable'); }
    public function videos()   { return $this->morphedByMany(Video::class,
'taggable'); }
}

// Article ও Video উভয় Model
```

```
class Article extends Model
{
    public function tags() { return $this->morphToMany(Tag::class, 'taggable'); }
}

// Article এ tags দেওয়া
$article = Article::find(1);
$article->tags()->attach([1, 2, 3]);

// Tag এর সব Articles
$tag      = Tag::find(1);
$articles = $tag->articles;
```

🎯 Practice করো

18. Post ও Video Model তৈরি করো যেখানে উভয়ের Comment আছে (morphMany)
19. 2টি Post এবং 2টি Video তৈরি করো, প্রতিটিতে 3টি comment করো
20. Comment থেকে commentable() দিয়ে parent বের করো
21. Post, Video উভয়ের জন্য Like system তৈরি করো (morphMany)

Module
07

Eager Loading ও N+1 Problem

Performance Killer সমস্যা ও সমাধান

with() | withCount | nested | lazy | conditional

📚 গল্প — Restaurant এ 100 জন Customer

ধরো একটি Restaurant এ 100 জন Customer বসা।

Waiter প্রতিজনকে জিজ্ঞেস করলো: 'আপনি কী খাবেন?'

তারপর Kitchen এ গেল একবার।

আবার ফিরে এলো, আবার একজনকে জিজ্ঞেস করলো।

এভাবে 100 বার Kitchen এ গেল — এটা কতটা inefficient←

সঠিক পথ: সবার order একসাথে নাও, একবার Kitchen এ যাও।

Lazy Loading = প্রতিজনের জন্য আলাদা trip (N+1 Problem)

Eager Loading = সবার order একসাথে, একটাই trip (Best Practice)

৭.১ N+1 Problem — বিস্তারিত

```
// ✗ N+1 PROBLEM — কখনো এভাবে করো না!
$posts = Post::all();   // Query 1: SELECT * FROM posts — ধরো 100টি post

foreach ($posts as $post) {
    echo $post->user->name;   // Query 2,3,4...101: প্রতিটির জন্য আলাদা!
    // 100টি post = 100টি extra query
    // Total: 1 + 100 = 101 queries!
}

// 1000 posts হলে = 1001 queries!
// Server slow হয়ে যাবে, user frustrated হবে।
```

⚠ **সাবধান:** N+1 Problem সবচেয়ে common Laravel performance issue। Production এ এটা server crash করাতে পারে। সবসময় Eager Loading ব্যবহার করো।

৭.২ Eager Loading — সমাধান

```
// ✓ EAGER LOADING — সঠিক পদ্ধতি
$posts = Post::with('user')->get();
// মাত্র 2টি query!
// Query 1: SELECT * FROM posts
// Query 2: SELECT * FROM users WHERE id IN (1,2,3,4...)

foreach ($posts as $post) {
    echo $post->user->name;   // কোনো extra query নেই!
}
```

৭.৩ Multiple Relationships Eager Loading

```
// একাধিক relationship একসাথে load
$posts = Post::with(['user', 'tags', 'comments'])->get();
// 4টি query: posts, users, tags, comments

// Nested relationship (dot notation)
$posts = Post::with('user.profile')->get();
// Post → User → Profile সব একসাথে

// তোমার E-commerce project এর মতো nested
$sales = Sale::with([
    'stock.product.brand',   // Sale → Stock → Product → Brand
    'stock.product.category',
    'user',
])->get();

foreach ($sales as $sale) {
    echo $sale->user->name;
    echo $sale->stock->product->name;
    echo $sale->stock->product->brand->name;
}
```

৭.৪ Conditional Eager Loading

```
// Condition সহ load
$users = User::with(['posts' => function($query) {
    $query->where('status', 'published')
          ->orderBy('created_at', 'desc')
          ->limit(5);         // প্রতি user এর সর্বশেষ ৫টি post
```

```
}])->get();

// Select specific columns (performance improve!)
$posts = Post::with('user:id,name,email')->get();
// শুধু user এর id, name, email load হবে

// Nested conditional
$users = User::with([
    'posts:id,user_id,title,status' => function($q) {
        $q->where('status', 'published');
    },
    'posts.comments:id,post_id,body',
])->get();
```

৭.৫ withCount, withSum, withAvg

```
// Posts count সহ Users
$users = User::withCount('posts')->get();
// $user->posts_count available

// Multiple counts
$users = User::withCount(['posts', 'comments', 'likes'])->get();

// Conditional count
$users = User::withCount(['posts as published_count' => function($q) {
    $q->where('status', 'published');
}])->get();
// $user->published_count

// Total order amount
$users = User::withSum('orders', 'amount')->get();
// $user->orders_sum_amount

// Average rating
$products = Product::withAvg('reviews', 'rating')->get();
// $product->reviews_avg_rating

// Min/Max
$products = Product::withMin('orders', 'price')
                   ->withMax('orders', 'price')
                   ->get();
```

৭.৬ Lazy Eager Loading (load)

```
// Data আগে load, relationship পরে load
$users = User::all();

// Later — condition পূরণ হলে তবে load করবো
if ($request->include_posts) {
    $users->load('posts');
}

if ($request->include_profile) {
    $users->load('profile');
}

// Conditional load
$users->load(['posts' => function($q) {
    $q->where('status', 'published');
}]);

// loadCount
$users->loadCount('posts');
$users->loadSum('orders', 'amount');
```

Method	কাজ	Query সংখ্যা
Post::with('user')->get()	Eager: সব এক request এ	2 (posts + users)
$post->user	Lazy: প্রথম access এ load	N+1 (loop এ)
$posts->load('user')	Lazy Eager: পরে load	+1 query
withCount('posts')	Count সহ parent load	2 queries
with('user:id,name')	Selective columns	2 queries (optimized)

🎯 Practice করো

22. 50টি Post তৈরি করো (Factory দিয়ে), Lazy Loading এ load করো এবং N+1 দেখো
23. তারপর Eager Loading দিয়ে same data load করো, query count compare করো
24. withCount দিয়ে প্রতিটি User এর post ও comment count বের করো
25. Nested eager loading: Post → User → Profile → City chain করো

Module
08

Querying Relationships

Relationship দিয়ে Filter করা

whereHas | doesntHave | withSum | Advanced

📚 গল্প — Library এ Book Search

একটি Library তে Books ও Authors আছে।

Librarian Sumon কে বললো:

'যে Authors এর কোনো Book নেই তাদের দেখাও'।

'যে Books এ কমপক্ষে 100টি Review আছে সেগুলো দেখাও'।

'English ভাষায় লেখা Book আছে এমন Authors দেখাও'।

Sumon Eloquent এর whereHas() ও has() ব্যবহার করে

এই সব complex queries সহজেই করলো।

৮.১ has() — Relationship আছে এমন records

```
// কমপক্ষে 1টি Book আছে এমন Authors
$authors = Author::has('books')->get();

// কমপক্ষে 5টি Book আছে এমন Authors
$authors = Author::has('books', '>=', 5)->get();

// Nested: Book আছে যে Book এর Review আছে এমন Authors
$authors = Author::has('books.reviews')->get();

// Exactly 3টি Book আছে
$authors = Author::has('books', '=', 3)->get();
```

৮.২ whereHas() — Condition সহ

```
// English ভাষায় Book আছে এমন Authors
$authors = Author::whereHas('books', function($q) {
    $q->where('language', 'English');
})->get();

// 100+ reviews এবং published আছে এমন Books
```

```
$books = Book::whereHas('reviews', function($q) {
    $q->where('rating', '>=', 4);
}, '>=', 100)->get();
// 'reviews' এ 100+ এমন records যেখানে rating >= 4

// Multiple whereHas
$authors = Author::whereHas('books', function($q) {
    $q->where('published', true);
})->whereHas('profile', function($q) {
    $q->where('country', 'Bangladesh');
})->get();

// orWhereHas
$users = User::whereHas('posts', function($q) {
    $q->where('status', 'published');
})->orWhereHas('comments', function($q) {
    $q->where('created_at', '>=', now()->subWeek());
})->get();
```

৮.৩ doesntHave() — Relationship নেই এমন records

```
// কোনো Book নেই এমন Authors
$noBookAuthors = Author::doesntHave('books')->get();

// Published Book নেই এমন Authors
$authors = Author::whereDoesntHave('books', function($q) {
    $q->where('published', true);
})->get();

// Orphan check — Profile নেই এমন Users
$usersWithoutProfile = User::doesntHave('profile')->get();
```

৮.৪ withAggregate — সব aggregate functions

```
// Sum
$users = User::withSum('orders', 'total_amount')->get();
echo $user->orders_sum_total_amount;

// Average
$products = Product::withAvg('reviews', 'rating')->get();
echo $product->reviews_avg_rating;

// Min & Max
```

```
$products = Product::withMin('orders', 'price')
                    ->withMax('orders', 'price')
                    ->get();


// Conditional aggregate
$users = User::withSum(['orders as completed_amount' => function($q) {
    $q->where('status', 'completed');
}], 'total_amount')->get();
echo $user->completed_amount;  // custom alias
```

🎯 Practice করো

26. কমপক্ষে 3টি Order দিয়েছে এমন Customers বের করো
27. সর্বোচ্চ 500 টাকার Order দিয়েছে এমন Customers বের করো (whereHas + wherePivot)
28. কোনো Order নেই এমন registered Users বের করো (doesntHave)
29. প্রতিটি Customer এর মোট খরচ (withSum) ও গড় order (withAvg) বের করো

Module
09

Accessors, Mutators ও Casts

Data পড়ার ও লেখার সময় Transform

get | set | $casts | Custom Cast

📚 গল্প — ATM মেশিন

ATM মেশিনে টাকা দেওয়ার সময় সেটা machine এর ভাষায় convert হয়।

তুলে নেওয়ার সময় আবার আমাদের ভাষায় convert হয়।

Mutator = টাকা জমার সময় convert (set করার সময়)

Accessor = টাকা তোলার সময় convert (get করার সময়)

উদাহরণ: User এর name সবসময় uppercase এ দেখাতে চাই।

তাহলে name এর Accessor তৈরি করবো।

বা password save হওয়ার সময় auto hash হবে।

তাহলে password এর Mutator তৈরি করবো।

৯.১ Accessor — পড়ার সময় Transform

```
use Illuminate\Database\Eloquent\Casts\Attribute;

class User extends Model
{
    // name সবসময় Title Case এ পাবে
    protected function name(): Attribute
    {
        return Attribute::make(
            get: fn($value) => ucwords(strtolower($value))
        );
    }

    // Phone number formatted দেখাবে: +880-1711-223344
    protected function phone(): Attribute
    {
        return Attribute::make(
            get: fn($value) => $value ? '+880-' . substr($value, 1, 4) . '-' .
substr($value, 5) : null
        );
```

```
    }

    // Virtual attribute — database এ নেই কিন্তু access করা যাবে
    protected function fullName(): Attribute
    {
        return Attribute::make(
            get: fn() => $this->first_name . ' ' . $this->last_name
        );
    }

    // Age calculate করা DOB থেকে
    protected function age(): Attribute
    {
        return Attribute::make(
            get: fn() => $this->dob ? $this->dob->diffInYears(now()) : null
        );
    }
}

// ব্যবহার
$user = User::find(1);
echo $user->name;      // 'Karim Hossain' (auto title case)
echo $user->full_name; // 'Karim Hossain' (virtual)
echo $user->age;       // 25 (calculated)
```

৯.২ Mutator — লেখার সময় Transform

```
class User extends Model
{
    // Password auto bcrypt হবে
    protected function password(): Attribute
    {
        return Attribute::make(
            set: fn($value) => bcrypt($value)
        );
    }

    // Email সবসময় lowercase এ save
    protected function email(): Attribute
    {
        return Attribute::make(
            get: fn($value) => strtolower($value),
            set: fn($value) => strtolower($value)  // save ও lowercase
        );
```

```
    }

    // Price — পয়সা হিসেবে save, টাকা হিসেবে দেখাও
    protected function price(): Attribute
    {
        return Attribute::make(
            get: fn($value) => $value / 100,      // পয়সা → টাকা
            set: fn($value) => $value * 100,      // টাকা → পয়সা
        );
    }
}

// ব্যবহার
$user = User::create(['password' => '123456']); // auto hash!
$user->email = 'KARIM@MAIL.COM';               // auto lowercase
$user->save();
```

৯.৩ Casts — Type Conversion

```
class Product extends Model
{
    protected $casts = [
        // Boolean
        'is_active'   => 'boolean',      // 0/1 → true/false
        'in_stock'    => 'boolean',

        // Numbers
        'price'       => 'decimal:2',    // 2 decimal places
        'rating'      => 'float',
        'stock_count' => 'integer',

        // Date/Time
        'launch_date'       => 'date',              // Carbon date
        'created_at'        => 'datetime',          // Carbon datetime
        'sale_ends_at'      => 'datetime:d M Y',    // Custom format

        // Array/JSON
        'specs'       => 'array',        // JSON string → PHP array
        'colors'      => 'collection',   // JSON → Collection
        'metadata'    => 'object',        // JSON → stdClass

        // Enum (PHP 8.1+)
        'status'      => ProductStatus::class,
    ];
```

```
}

// ব্যবহার
$product = Product::find(1);
if ($product->is_active) {          // Boolean check — সুন্দর!
    $specs = $product->specs;        // Array automatically
    echo $specs['weight'];
}
echo $product->launch_date->format('d M, Y'); // Carbon object
```

📌 **নোট:** Virtual attribute (full_name) কে JSON/Array তে include করতে হলে Model এ $appends = ['full_name'] যোগ করো।

Module
10

Scopes

Query Reuse ও Chain

Local Scope | Global Scope | Named Scope | withoutScope

📚 গল্প — রান্নার Recipe Book

একজন Chef আছেন যিনি প্রতিদিন একই sauce বানান।

প্রতিদিন একই ingredients বের করেন, একই steps follow করেন।

Scope হলো সেই saved recipe — একবার লিখলেই যেকোনো জায়গায় ব্যবহার করা যায়।

যেমন: 'active users' মানে where status=1 — এটা সবখানে লাগে।

Local Scope তৈরি করলে শুধু active() লিখলেই হয়।

Global Scope মানে সবসময় automatically সেই condition apply হবে।

Soft Delete এর deleted_at IS NULL এর মতো।

১০.১ Local Scope — চাহিদামতো ব্যবহার

```
class Product extends Model
{
    // scopeActive → Product::active()
    public function scopeActive($query)
    {
        return $query->where('is_active', true);
    }

    // scopeInStock → Product::inStock()
    public function scopeInStock($query)
    {
        return $query->where('stock_quantity', '>', 0);
    }

    // Parameter সহ scope
    public function scopeByCategory($query, $categoryId)
    {
        return $query->whereHas('categories', fn($q) => $q->where('id',
$categoryId));
    }
```

```
    public function scopePriceBetween($query, $min, $max)
    {
        return $query->whereBetween('price', [$min, $max]);
    }

    public function scopeSearch($query, $term)
    {
        return $query->where('name', 'LIKE', "%{$term}%")
                     ->orWhere('description', 'LIKE', "%{$term}%");
    }
}

// ব্যবহার — Chain করা যায়!
$products = Product::active()->inStock()->get();
$products = Product::active()->priceBetween(100, 500)->get();
$products = Product::search('laptop')->active()->inStock()->paginate(20);
$products = Product::byCategory(3)->active()->inStock()->orderBy('price')->get();
```

১০.২ Global Scope — সবসময় Apply

```
// একটি Scope class তৈরি করো
php artisan make:scope ActiveScope

// app/Models/Scopes/ActiveScope.php
use Illuminate\Database\Eloquent\{Builder, Model, Scope};

class ActiveScope implements Scope
{
    public function apply(Builder $builder, Model $model): void
    {
        $builder->where('is_active', true);
    }
}

// Product Model এ Global Scope register করো
class Product extends Model
{
    protected static function booted(): void
    {
        static::addGlobalScope(new ActiveScope());
        // এখন Product::all() সবসময় শুধু active products দেবে!
    }
}
```

```
// Global scope bypass করতে:
$allProducts = Product::withoutGlobalScope(ActiveScope::class)->get();
$allProducts = Product::withoutGlobalScopes()->get(); // সব global scopes ignore
```

Module
11

Soft Deletes

Data না মুছে Archive করা

SoftDeletes | withTrashed | onlyTrashed | restore | forceDelete

📚 গল্প — Recycle Bin

Windows এ কোনো file delete করলে সেটা Recycle Bin এ যায়।
সত্যিকারে মুছে যায় না।পরে দরকার হলে restore করা যায়।

Soft Delete ঠিক এটাই করে←

একটি Order delete করলে, database থেকে আসলে মুছে না।
শুধু deleted_at column এ time set হয়।
সাধারণ query তে সে দেখা যায় না।
কিন্তু চাইলে restore করা যায়।

E-commerce এ Order, User, Product কখনোই permanently delete করা উচিত না।
Audit trail থাকা দরকার।Soft Delete সেটাই দেয়।

১১.১ Setup

```
// Migration এ deleted_at column যোগ করো
Schema::table('orders', function (Blueprint $table) {
    $table->softDeletes(); // deleted_at TIMESTAMP nullable
});

// Order Model এ trait যোগ করো
use Illuminate\Database\Eloquent\SoftDeletes;

class Order extends Model
{
    use SoftDeletes;

    // এখন Order::find(1)->delete() করলে deleted_at set হবে, মুছবে না!
}
```

১১.২ Soft Delete Operations — সব Methods

```
// Soft Delete করা
$order = Order::find(1);
$order->delete();   // deleted_at = now() — আসলে মুছে না

// Bulk soft delete
Order::where('status', 'cancelled')->delete();

// ─── Check ──────────────────────────────────
if ($order->trashed()) {
    echo 'Order টি delete করা হয়েছে';
}

// ─── Retrieve ───────────────────────────────
// শুধু active (default behavior)
$orders = Order::all();

// Active + Deleted সবকিছু
$all = Order::withTrashed()->get();

// শুধু Deleted গুলো (Recycle Bin)
$deleted = Order::onlyTrashed()->get();
$deleted = Order::onlyTrashed()->orderBy('deleted_at', 'desc')->get();

// ─── Restore ────────────────────────────────
Order::withTrashed()->find(1)->restore();

// Bulk restore
Order::withTrashed()->where('user_id', 5)->restore();

// ─── Force Delete (চিরতরে মুছে ফেলা) ──────────
Order::withTrashed()->find(1)->forceDelete();

// 30 দিনের বেশি পুরনো deleted orders চিরতরে মুছো
Order::onlyTrashed()
      ->where('deleted_at', '<', now()->subDays(30))
      ->forceDelete();
```

১১.৩ Relationship ও Soft Delete

```
// User delete হলে তার সব Posts ও soft delete করো
class User extends Model
```

```
{
    use SoftDeletes;

    protected static function booted()
    {
        // User delete হলে
        static::deleted(function($user) {
            $user->posts()->delete();  // Posts ও soft delete
        });

        // User restore হলে
        static::restored(function($user) {
            $user->posts()->restore(); // Posts ও restore
        });
    }
}

// Soft deleted posts ও include করতে
$user = User::withTrashed()->with(['posts' => function($q) {
    $q->withTrashed();
}])->find(1);
```

Module
12

Model Events ও Observers

Model Lifecycle | Auto Actions | Clean Code

creating | created | updating | deleted | Observer class

📚 গল্প — Security Camera System

একটি Bank এ Security Camera আছে।

কেউ enter করলে automatically camera record করে।

কেউ টাকা তুললে alert যায়।

কেউ logout করলে log entry হয়।

Model Events ঠিক এটাই করে←

User create হলে — welcome email পাঠাও (created event).

Product delete হলে — inventory update করো (deleting event).

Order status change হলে — SMS পাঠাও (updated event).

এগুলো automatically হয় — controller এ code লিখতে হয় না।

১২.১ Available Events

Event	কখন fire হয়	সাধারণ ব্যবহার
creating	save() এর আগে (নতুন)	UUID/slug auto generate
created	save() এর পরে (নতুন)	Welcome email, notification
updating	save() এর আগে (পুরনো)	Validation, old value capture
updated	save() এর পরে (পুরনো)	Audit log, cache clear
saving	create বা update উভয়ের আগে	Common validation
saved	create বা update উভয়ের পরে	Cache refresh
deleting	delete() এর আগে	Related data cleanup
deleted	delete() এর পরে	Log entry, notification
restoring	restore() এর আগে	Validation
restored	restore() এর পরে	Reactivate related data

১২.২ Model এ সরাসরি Events

```
class Product extends Model
{
    use SoftDeletes;

    protected static function booted(): void
    {
        // Product create হওয়ার আগে slug তৈরি করো
        static::creating(function($product) {
            $product->slug = Str::slug($product->name) . '-' . time();
            $product->sku  = 'PRD-' . strtoupper(Str::random(8));
        });

        // Product update হলে cache clear করো
        static::updated(function($product) {
            Cache::forget('product_' . $product->id);
            Cache::forget('featured_products');
        });

        // Product delete হলে images delete করো
        static::deleting(function($product) {
            foreach ($product->images as $image) {
                Storage::delete($image->url);  // Physical file delete
                $image->delete();              // DB record delete
            }
        });
    }
}
```

১২.৩ Observer Class — পরিচ্ছন্ন পদ্ধতি

```
php artisan make:observer OrderObserver --model=Order

// app/Observers/OrderObserver.php
class OrderObserver
{
    public function creating(Order $order): void
    {
        // Order number auto generate
        $order->order_number = 'ORD-' . date('Ymd') . '-' .
strtoupper(Str::random(6));
    }

    public function created(Order $order): void
```

```
    {
        // Customer কে confirmation email
        Mail::to($order->user->email)->send(new OrderConfirmationMail($order));

        // Admin কে notification
        Notification::send(User::admins()->get(), new
NewOrderNotification($order));
    }

    public function updated(Order $order): void
    {
        // Status change হলে Customer কে SMS
        if ($order->isDirty('status')) {
            $oldStatus = $order->getOriginal('status');
            $newStatus = $order->status;
            // Send SMS...
        }
    }

    public function deleted(Order $order): void
    {
        // Order delete হলে stock ফেরত দাও
        foreach ($order->items as $item) {
            $item->product->increment('stock_quantity', $item->quantity);
        }
    }
}

// AppServiceProvider.php এ register করো
use App\Models\Order;
use App\Observers\OrderObserver;

class AppServiceProvider extends ServiceProvider
{
    public function boot(): void
    {
        Order::observe(OrderObserver::class);
    }
}
```

💡 **isDirty()** $model->isDirty('status') — field টি change হয়েছে কিনা check করে। getOriginal('status') — পুরনো value দেয়। getChanges() — সব changed fields দেয়।

Module
13

Factory ও Seeder

Dummy Data তৈরি করা

Factory | State | Seeder | Faker | Relationship Data

📚 গল্প — Testing এর জন্য নকল Data

Nadia একটি E-commerce site test করছে।

তার 1000টি Product, 500 জন User এবং 2000টি Order দরকার।

হাতে হাতে করা সম্ভব নয়।

তার colleague বললো: Factory ও Seeder ব্যবহার করো←

Factory হলো blueprint — Model এর fake data কেমন হবে।

Seeder হলো runner — কতটা data তৈরি করবে।

Faker library দিয়ে realistic fake data তৈরি হয়:

fake()->name() — 'Md. Karim Hossain'

fake()->email() — 'karim.hossain@gmail.com'

fake()->phoneNumber() — '+8801711223344'

১৩.১ Factory তৈরি

```
php artisan make:factory ProductFactory --model=Product

// database/factories/ProductFactory.php
class ProductFactory extends Factory
{
    protected $model = Product::class;

    public function definition(): array
    {
        $name  = fake()->words(3, true);
        return [
            'brand_id'       => Brand::factory(),  // auto Brand তৈরি করবে
            'name'           => ucfirst($name),
            'slug'           => Str::slug($name) . '-' .
fake()->unique()->randomNumber(5),
            'description'    => fake()->paragraphs(2, true),
            'price'          => fake()->randomFloat(2, 10, 5000),
```

```
            'sale_price'     => fake()->optional(0.3)->randomFloat(2, 5, 4000),
            'stock_quantity' => fake()->numberBetween(0, 500),
            'is_active'      => fake()->boolean(80), // 80% chance true
        ];
    }

    // State: Out of stock product
    public function outOfStock(): static
    {
        return $this->state(['stock_quantity' => 0, 'is_active' => false]);
    }

    // State: Featured product
    public function featured(): static
    {
        return $this->state([
            'is_featured'    => true,
            'stock_quantity' => fake()->numberBetween(50, 500),
            'is_active'      => true,
        ]);
    }

    // State: Sale product
    public function onSale(): static
    {
        return $this->state(function(array $attributes) {
            return ['sale_price' => $attributes['price'] * 0.7]; // 30% off
        });
    }
}
```

১৩.২ Factory ব্যবহার করা

```
// ১টি Product
$product = Product::factory()->create();

// 50টি Product
Product::factory()->count(50)->create();

// State ব্যবহার
Product::factory()->outOfStock()->count(10)->create();
Product::factory()->featured()->count(5)->create();
Product::factory()->onSale()->count(20)->create();
```

```
// Relationship সহ (Brand ও auto তৈরি হবে)
Product::factory()->for(Brand::factory())->create();

// নির্দিষ্ট brand এ products
$brand = Brand::first();
Product::factory()->for($brand)->count(10)->create();

// Many-to-Many সহ
$product = Product::factory()
    ->hasAttached(
        Category::factory()->count(3),
        ['created_at' => now()]
    )->create();
```

১৩.৩ Seeder

```
php artisan make:seeder DatabaseSeeder

// database/seeders/DatabaseSeeder.php
class DatabaseSeeder extends Seeder
{
    public function run(): void
    {
        // ─── Static Data (পরিবর্তন হয় না) ──────────

        // Brands
        $brands = ['Samsung', 'Apple', 'Sony', 'LG', 'HP']
            |> array_map(fn($b) => Brand::create(['name' => $b, 'slug' =>
Str::slug($b), 'is_active' => true]));

        // Categories
        $categories = ['Electronics', 'Mobile', 'Laptop', 'TV', 'Accessories']
            |> collect()->each(fn($c) => Category::create(['name' => $c, 'slug'
=> Str::slug($c)]));

        // ─── Dynamic Data (Factory দিয়ে) ──────────

        // 500 Users
        $users = User::factory()->count(500)->create();

        // 200 Products (with Brand)
        $products = Product::factory()->count(200)->create();

        // প্রতিটি Product এ random categories attach
```

```
        $products->each(function($product) use ($categories) {
            $product->categories()->attach($categories->random(rand(1,
3))->pluck('id'));
        });


        // প্রতিটি User এর 1-5টি Orders
        $users->each(function($user) use ($products) {
            $orderCount = rand(1, 5);

Order::factory()->count($orderCount)->for($user)->create()->each(function($order)
use ($products) {
                // প্রতিটি Order এ 1-4টি Items
                $items = $products->random(rand(1, 4));
                foreach ($items as $product) {
                    $qty = rand(1, 5);
                    $order->items()->create([
                        'product_id' => $product->id,
                        'quantity'   => $qty,
                        'price'      => $product->price,
                        'total'      => $product->price * $qty,
                    ]);
                }
            });
        });

        // Admin User
        User::factory()->create([
            'name'     => 'Admin User',
            'email'    => 'admin@admin.com',
            'password' => bcrypt('password'),
            'role'     => 'admin',
        ]);
    }
}


// Run করার commands
php artisan db:seed
php artisan db:seed --class=DatabaseSeeder
php artisan migrate:fresh --seed  // Fresh migrate + seed
```

Module
14

API Resource

JSON Response Control করা

JsonResource | Collection | whenLoaded | conditional

📚 গল্প — Bank Statement

তুমি Bank Statement চাইলে Bank তোমাকে শুধু তোমার লেনদেনের তথ্য দেয়।

অন্য সব customers এর তথ্য দেয় না।

Password, Internal notes কিছুই দেয় না।

API Resource ঠিক এটাই করে←

Database Model এ অনেক sensitive data থাকতে পারে।

API Resource দিয়ে নির্দিষ্ট করা যায় কোন fields JSON এ যাবে।

পাশাপাশি relationship data formatted করে পাঠানো যায়।

১৪.১ Resource তৈরি ও ব্যবহার

```
php artisan make:resource UserResource
php artisan make:resource ProductResource

// app/Http/Resources/UserResource.php
class UserResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        return [
            'id'          => $this->id,
            'name'        => $this->name,
            'email'       => $this->email,
            'phone'       => $this->profile?->phone,
            'city'        => $this->profile?->city,
            'role'        => $this->role,
            'posts_count' => $this->posts_count ?? 0,
            'joined_date' => $this->created_at->format('d M, Y'),

            // Conditional: relationship load হলেই include
            'profile' => ProfileResource::make($this->whenLoaded('profile')),
            'posts'   => PostResource::collection($this->whenLoaded('posts')),
```

```
            'orders'  => OrderResource::collection($this->whenLoaded('orders')),


            // Conditional: admin হলেই দেখাবে
            'internal_notes' => $this->when(auth()->user()?->isAdmin(),
$this->notes),
        ];
    }
}


// Controller এ ব্যবহার
class UserController extends Controller
{
    public function index()
    {
        $users = User::with('profile')->withCount('posts')->paginate(20);
        return UserResource::collection($users);
    }

    public function show(User $user)
    {
        $user->load(['profile', 'posts' => fn($q) => $q->latest()->limit(10)]);
        return UserResource::make($user);
    }
}
```

Module
15

Real Project: ShopBD

সম্পূর্ণ E-Commerce System

Product | Category | Brand | Order | Cart | Review | Full API

📚 ShopBD — Project Background

আমরা 'ShopBD' নামে একটি E-Commerce platform বানাবো।

Features:

- Brand ও Category দিয়ে Products organize করা
- User Registration ও Authentication
- Shopping Cart system
- Order placement ও tracking
- Product Review ও Rating
- Admin dashboard

এই project এ সব ধরনের Relationship থাকবে:

One-to-One, One-to-Many, Many-to-Many, Polymorphic, Soft Delete, Observer

১৫.১ Database Design

Table	Columns (key)	Relationship
brands	id, name, slug, logo, is_active	→ products (One-to-Many)
categories	id, parent_id, name, slug	→ products (Many-to-Many), self-ref
products	id, brand_id, name, slug, price, stock	← brand, ↔ categories, → reviews/images
users	id, name, email, password, role	→ orders, profile, cart
profiles	id, user_id, phone, avatar, address	← user (One-to-One)
carts	id, user_id, product_id, quantity	← user, ← product
orders	id, user_id, order_number, status, total	← user, → order_items
order_items	id, order_id, product_id, qty, price	← order, ← product
reviews	id, user_id, product_id, rating, body	← user, ← product
images	id, url, imageable_id, imageable_type	← products/categories (Poly)

১৫.২ Core Models

```
// app/Models/Product.php
```

```
class Product extends Model
{
    use SoftDeletes, HasFactory;


    protected $fillable =
['brand_id','name','slug','description','price','sale_price','stock_quantity','is_activ
e'];


    protected $casts = [
        'price'       => 'decimal:2',
        'sale_price'  => 'decimal:2',
        'is_active'   => 'boolean',
    ];


    // ─── Relationships ──────────────────────────
    public function brand()       { return $this->belongsTo(Brand::class); }
    public function categories()  { return $this->belongsToMany(Category::class,
'category_product'); }
    public function reviews()     { return $this->hasMany(Review::class); }
    public function images()      { return $this->morphMany(Image::class,
'imageable'); }
    public function orderItems()  { return $this->hasMany(OrderItem::class); }


    // ─── Scopes ─────────────────────────────────
    public function scopeActive($q)         { return $q->where('is_active', true); }
    public function scopeInStock($q)        { return $q->where('stock_quantity', '>',
0); }
    public function scopeFeatured($q)       { return $q->where('is_featured', true); }
    public function scopeSearch($q, $term)  { return
$q->where('name','LIKE',"%{$term}%"); }
    public function scopeByBrand($q, $id)   { return $q->where('brand_id', $id); }


    // ─── Accessors ──────────────────────────────
    protected function effectivePrice(): Attribute {
        return Attribute::make(
            get: fn() => $this->sale_price ?? $this->price
        );
    }
    protected function discountPercent(): Attribute {
        return Attribute::make(
            get: fn() => $this->sale_price
                ? round((1 - $this->sale_price / $this->price) * 100)
                : 0
        );
    }


    // ─── Events ─────────────────────────────────
```

```
    protected static function booted() {
        static::creating(fn($p) => $p->slug = Str::slug($p->name) . '-' . time());
    }
}
```

```
// app/Models/Order.php
class Order extends Model
{
    use SoftDeletes, HasFactory;

    protected $fillable =
['user_id','order_number','status','subtotal','discount','shipping','total','address'];

    protected $casts = [
        'subtotal' => 'decimal:2',
        'total'    => 'decimal:2',
    ];

    public function user()   { return $this->belongsTo(User::class); }
    public function items()  { return $this->hasMany(OrderItem::class); }

    public function scopePending($q)    { return $q->where('status','pending'); }
    public function scopeCompleted($q)  { return $q->where('status','delivered'); }

    protected static function booted() {
        static::creating(fn($o) => $o->order_number = 'SBD-' . date('Ymd') . '-' .
strtoupper(Str::random(6)));
    }
}
```

১৫.৩ Order Controller — Full Example

```
class OrderController extends Controller
{
    // Order তৈরি করা (Checkout)
    public function store(CheckoutRequest $request)
    {
        return DB::transaction(function() use ($request) {
            $cart = Cart::where('user_id', auth()->id())
                        ->with('product')
                        ->get();

            if ($cart->isEmpty()) {
                return response()->json(['message' => 'Cart empty'], 422);
```

```
            }

            // Stock check
            foreach ($cart as $item) {
                if ($item->product->stock_quantity < $item->quantity) {
                    return response()->json([
                        'message' => $item->product->name . ' এর stock নেই',
                    ], 422);
                }
            }

            // Subtotal calculate
            $subtotal = $cart->sum(fn($i) => $i->product->effective_price *
$i->quantity);
            $shipping = $subtotal >= 500 ? 0 : 60;  // Free shipping > 500
            $total    = $subtotal + $shipping;

            // Order create
            $order = Order::create([
                'user_id'  => auth()->id(),
                'subtotal' => $subtotal,
                'shipping' => $shipping,
                'total'    => $total,
                'address'  => $request->address,
                'status'   => 'pending',
            ]);

            // Order items create ও stock কমানো
            foreach ($cart as $item) {
                $order->items()->create([
                    'product_id' => $item->product_id,
                    'quantity'   => $item->quantity,
                    'price'      => $item->product->effective_price,
                    'total'      => $item->product->effective_price *
$item->quantity,
                ]);

                // Stock কমাও
                $item->product->decrement('stock_quantity', $item->quantity);
            }

            // Cart clear করো
            Cart::where('user_id', auth()->id())->delete();

            return new OrderResource($order->load('items.product.brand'));
```

```
        });
    }

    // Order list (User এর নিজেরটা)
    public function index()
    {
        $orders = Order::where('user_id', auth()->id())
                        ->with(['items.product:id,name,price'])
                        ->withCount('items')
                        ->latest()
                        ->paginate(10);

        return OrderResource::collection($orders);
    }
}
```

Module
16

Best Practices ও Cheat Sheet

পরিষ্কার Code | Performance | Common Mistakes

Naming | Query Tips | Anti-patterns | Quick Reference

১৬.১ Naming Convention

কী	Convention	উদাহরণ
Model	PascalCase Singular	User, Product, OrderItem
Table	snake_case Plural	users, products, order_items
Migration	action_tablename_table	create_users_table, add_status_to_orders_table
Foreign Key	singular_model_id	user_id, product_id, brand_id
Pivot Table	alpha singular models	post_tag, category_product
Pivot Column	descriptive	created_by, approved_at, quantity
Relationship Method	camelCase	posts(), orderItems(), latestOrder()
Scope	scopeCamelCase	scopeActive(), scopeByUser()
Observer	ModelObserver	UserObserver, OrderObserver
Resource	ModelResource	UserResource, ProductResource

১৬.২ Performance Quick Reference

সমস্যা	খারাপ পদ্ধতি	ভালো পদ্ধতি
N+1	Post::all() + loop $p->user	Post::with('user')->get()
All Columns	User::with('posts')->get()	User::select('id','name')->with('posts:id,user_id,title')
Count in Loop	$user->posts->count()	User::withCount('posts') → $user->posts_count
All Data	User::all()	User::paginate(20)
No Index	WHERE email='...' (no index)	$table->index('email')
Boolean Check	where('status','=',1)	where('status', true) বা scope
Duplicate Query	Same query বারবার	Cache::remember('key', 60, fn() => ...)
Eager ও Lazy Mix	load() loop এ	with() eager loading

১৬.৩ Common Anti-patterns

⚠ **সাবধান:** এই ভুলগুলো Production এ কখনো করো না:

```
// ✗ Anti-pattern 1: N+1 in loop
foreach (Post::all() as $post) { echo $post->user->name; }
// ✓ Fix:
foreach (Post::with('user')->get() as $post) { echo $post->user->name; }

// ✗ Anti-pattern 2: Unnecessary all columns
$users = User::with('posts')->all();
// ✓ Fix:
$users = User::select('id','name')->with('posts:id,user_id,title')->paginate(20);

// ✗ Anti-pattern 3: DB query in Blade template
// blade: @foreach(User::all() as $user) — এটা View এ query করা!
// ✓ Fix: Controller এ query করে pass করো

// ✗ Anti-pattern 4: No transaction for related operations
Order::create([...]);
foreach($items as $item) { OrderItem::create([...]); }
// ✓ Fix: DB::transaction() এর ভেতরে

// ✗ Anti-pattern 5: Count with relationship load
$count = $user->posts->count();   // সব posts load করে তারপর count!
// ✓ Fix:
$count = $user->posts()->count(); // শুধু COUNT query
```

১৬.৪ Final Cheat Sheet

Relationship	Model A Method	Model B Method	Pivot?
One-to-One	hasOne(B::class)	belongsTo(A::class)	না
One-to-Many	hasMany(B::class)	belongsTo(A::class)	না
Many-to-Many	belongsToMany(B::class)	belongsToMany(A::class)	হ্যাঁ
Has One Through	hasOneThrough(C::class, B::class)	—	না
Has Many Through	hasManyThrough(C::class, B::class)	—	না
Morph One	morphOne(B::class,'able')	morphTo()	না
Morph Many	morphMany(B::class,'able')	morphTo()	না
Morph Many-to-Many	morphToMany(B::class,'able')	morphedByMany(A::class,'able')	হ্যাঁ

অভিনন্দন← তুমি বইটি শেষ করেছো।

এখন তোমার কাজ হলো প্রতিটি Module এর Practice গুলো করা।Real Project বানানো ছাড়া শেখা সম্পূর্ণ হয় না।

Happy Coding! 🚀